

QWEN

Helix Cluster OS — QWEN.md

Field	Value
Revision	2
Created	2026-05-30
Last modified	2026-06-02T09:49:46Z
Status	active

INHERITED FROM HelixConstitution/QWEN.md

All rules in HelixConstitution/QWEN.md (and the HelixConstitution/Constitution.md, HelixConstitution/CLAUDE.md, and HelixConstitution/AGENTS.md it references) apply unconditionally. The project-specific rules below extend them.

Project-Specific Extensions

Agent Context

- This is the Helix Cluster OS — a distributed cluster operating system.
- Seven-layer stack (L0-L7), 14 control plane microservices, SWIM gossip + Raft consensus.
- Cross-platform parity mandatory: Linux, macOS, Windows/WSL2 per-feature.
- GPU backend coverage mandatory: NVIDIA CUDA, AMD ROCm, Intel oneAPI, Apple MLX on real hardware.

CodeGraph Integration

- CodeGraph is installed, indexed, and wired via MCP.
- Qwen Code SHOULD use CodeGraph for symbol resolution and cross-reference queries.
- See docs/CODEGRAPH.md for setup and usage.

Critical Base Rules Restated

- Anti-bluff covenant — END-USER QUALITY GUARANTEE (§11.4).
- Mutation-paired gates (§1.1).
- Credentials-handling mandate (§11.4.10) + pre-store leak audit (§11.4.10.A).
- Subagent-driven-by-default (§11.4.20).
- Fetch-before-edit (§11.4.37).
- CodeGraph code-intelligence mandate (§11.4.78) — installed and active.

QWEN-1: End-User Usability Guarantee (§7.1 + §11.4.39 Reinforcement)

Forensic anchor — verbatim user mandate: > “We had been in position that all tests do execute with success and all Challenges as well, but in reality the most of the features does not work and can’t be used! This MUST NOT be the case and execution of tests and Challenges MUST guarantee the quality, the completion and full usability by end users of the product!”

Operative rules for QA agents: 1. **Tests are necessary but NOT sufficient.** Every feature under QA MUST be validated with: - Unit tests (with paired mutation per §1.1) - Integration tests against REAL services (not mocks) - End-to-end tests that exercise the feature as an end user would - Challenge validation via HelixQA framework 2. **A test that passes on a non-functional feature is a PASS-bluff** — severity equivalent to §7.1 violation. You MUST file a defect and block release. 3. **Every test MUST prove the feature works for end users**, not just that code executes without panic. Validate actual user journeys and observable outcomes. 4. **Challenges (HelixQA) are bound equally** — a Challenge PASS on a broken feature is the same class of defect as a unit test PASS on broken code. Treat both with equal severity. 5. **No mock-only validation** for features that claim real-world operation. Mocks are permitted ONLY in unit tests per §11.4.27. Reject any feature validated solely with mocks. 6. **Sink-side evidence required:** Every feature closure MUST include captured evidence (screenshot, log, metrics) proving end-user-visible operation. Your sign-off is contingent on this evidence.

QWEN-2: Documentation & Materials Continuous-Sync Guarantee (§11.4.106 docs-chain + family)

Forensic anchor — verbatim user mandate: > “while we continuously change the project ... whatever we do on project or any of its services, components, architecture or anything else MUST trigger proper updateing of main README document, all project documentation, user guides, manuals, webiste(s) (if any), diagrams, graphs, schemes, SQL defintions and all other related materials! ... Connect all this with docs_chain Submodule and make sure that all this is achieved out of the box, never forgotten or skipped and all docs and materials maintained regularly!”

Restates + cites the binding master rule (Constitution §11.4.106 docs-chain-mechanical-sync + anchor family §11.4.12/45/53/56/57/59/60/65/86/93/95, §12.10, §11.4.44; “No documentation ever can be out of sync with its codebase”). Per §11.4.35 inheritance.

Operative rules for QA agents: 1. **Block release on stale docs.** A change that ships code/service/component/architecture/schema/script changes without updating every affected material — main README.md, all docs/**, user guides, manuals,

website(s), diagrams/graphs/schemes, SQL/schema definitions, and ALL their exports (md → html/pdf/docx) — is a §11.4.106 violation; file a defect and block. 2. **Verify via docs_chain.** Gate on docs_chain verify (deterministic sink-side check over byte-stable transforms); confirm new materials are registered in .docs_chain/contexts/*.yaml so enforcement is automatic and out of the box. Reject hand-maintained exports. 3. **No fake PASS.** A metadata-only / absence-of-error / config-only PASS at the sync layer is a §11.4 PASS-bluff; require real captured evidence at qa-results/docs_chain/<run-id>/. 4. **Continuous maintenance** — documentation sync is validated every wave, not just at release; treat a stale doc with the same severity as a failing test.